

Otimização de promoções em loja de conveniência com Reinforcement Learning

Vinicius Rocha^a and Luigi Zema^a

^aInsper Instituto de Ensino e Pesquisa

Prof. Fabrício Barth

Resumo—Lojas de conveniência de postos de combustível perdem margem com promoções mal escolhidas e produtos vencidos. Modelamos a decisão promocional do *Auto Posto Parque Viana* (Barueri/SP) como um MDP e treinamos um agente *Branching Double DQN* num ambiente *Gymnasium* próprio, calibrado com seis anos de vendas reais e uma matriz de afinidade construída a partir do conhecimento do balcão de posto (sem dados de supermercado). O agente decide, a cada turno, três coisas em conjunto: intensidade do desconto, produto complementar do combo e em qual item aplicar o desconto. Após 5000 episódios, a curva de aprendizado converge (volatilidade −88%) e o agente acerta 11/12 cenários canônicos, gera um calendário anual de 208 campanhas sem nenhum combo inválido para o contexto de posto, e faz emergir sozinho padrões como a “Esquenta de Sexta” (cerveja + salgadinho). Reportamos abertamente a limitação central: a elasticidade é da literatura, não medida no posto.

keywords—Reinforcement Learning, Double DQN, Branching DQN, reward shaping, varejo, precificação promocional

1. Introdução

O *Auto Posto Parque Viana*, em Barueri/SP, opera uma loja de conveniência cujas promoções são decididas hoje por intuição do dono. O problema tem custo real: desconto em produto que já vende sozinho destrói margem; produto sem giro vence e vira prejuízo; oportunidades sazonais (Copa, Dia dos Namorados, fim de semana) são perdidas. O **objetivo** é maximizar o lucro anual da conveniência minimizando perda por vencimento, entregando um calendário operacional de campanhas (data, produto, combo, desconto).

A decisão é **sequencial e sob incerteza**: o estado da loja evolui (estoque, validade, dia, evento próximo, clima), cada decisão altera o estado seguinte e existe *trade-off* entre lucro imediato e prevenção de vencimento futuro. Isso é exatamente a estrutura de um *Markov Decision Process*; supervisão pura não captura o efeito da ação sobre o estado futuro nem o horizonte longo. Por isso usamos *Reinforcement Learning*: a função de recompensa codifica o lucro econômico real e o agente aprende, por tentativa e erro em milhares de dias simulados, quando e o que promover.

2. Ambiente

O ambiente é um *Gymnasium* implementado pela equipe (EnvRLLPromocoos), calibrado com dados reais do posto: seis anos de vendas por categoria e turno, preços/custos/margens por SKU, descarte por vencimento, inventário 2022–2026 (12.750 registros, 87 classificações reduzidas a 23 categorias de conveniência após remover itens automotivos) e temperatura histórica de Barueri.

Estado — um vetor contínuo de 89 *features*. A escolha não é uma imagem nem um vetor abstrato: cada bloco corresponde a uma informação que o dono de fato usa para decidir. São eles: turno (*one-hot*, 3), dia da semana (7), mês (12), proximidade e tipo do próximo evento comercial, temperatura normalizada, e — por categoria — estoque relativo, validade restante, *flag* de demanda historicamente fraca e histórico de promoção recente. Representar o estado *por categoria* é o que permite ao agente diferenciar “gelo no sábado de verão” de “vinho numa terça” — a granularidade segue a estrutura econômica do problema, não uma conveniência de implementação.

Ações — discretas e fatoradas como `MultiDiscrete([6, 24, 2])`, isto é, três decisões simultâneas tomadas a cada turno (Tabela 1). Fatorar evita a explosão combinatória ($6 \times 24 \times 2 = 288$ ações conjuntas viram $6 + 24 + 2 = 32$ saídas). O ambiente é **estocástico**: a demanda realizada é $q_i \sim \text{Poisson}(\lambda_i)$.

Tabela 1. Espaço de ações fatorado (três decisões por turno).

Cabeça	Papel	Opções
Intensidade	Agente de Desconto	nada, 3%, 5%, 7%, 10%, combo
Complementar	Agente de Combo	nenhum + 23 categorias
Alvo	Agente de Margem	desconto no principal/par

Recompensa — o núcleo é o *lucro real sobre a demanda promocional*, não sobre a demanda base. A promoção altera a demanda (Eq. 1); o lucro (Eq. 2) soma o ganho do *uplift*, subtrai canibalização, soma *halo* (venda casada) e o ganho defensivo de liquidar item prestes a vencer.

$$d_i^{\text{promo}} = \underbrace{d_i^{\text{base}} \cdot f_{\text{saz}}(t)}_{d_i^{\text{ctx}}} (1 + \text{uplift}_i(a)) \quad (1)$$

$$r_t = \underbrace{L_{\text{uplift}} + L_{\text{halo}} + L_{\text{canib}} + L_{\text{def}}}_{\text{lucro econômico}} + \sum_k \kappa_k \phi_k(s_t, a_t) \quad (2)$$

O segundo termo é *reward shaping* de domínio (≈ 38 componentes κ_k): premia harmonia de combo válida no posto, acerto de evento comercial, prevenção de vencimento e desconto em produto de baixo giro; penaliza desconto em produto que já está em alta natural e combos inválidos para posto (ex.: gelo + whisky — comportamento de supermercado, não de balcão de conveniência).

3. Método

Algoritmo

Usamos *Branching Double DQN* — a arquitetura *action branching* de Tavakoli et al. [5] sobre o DQN [3] com o alvo *Double DQN* [4], e *reward shaping* no sentido formal de Ng, Harada & Russell [1].

A rede tem um *encoder* compartilhado ($89 \rightarrow 128 \rightarrow 128$, ReLU) e **três cabeças independentes** ($128 \rightarrow 64 \rightarrow n$), uma por decisão da Tabela 1. As cabeças partilham a mesma leitura do estado mas têm *Q-values* próprios, treinadas pela soma das perdas (*Huber*). Conceitualmente são três sub-agentes coordenados (Desconto, Combo, Margem) — forma reconhecida de RL multi-cabeça coordenado.

A **implementação é própria**, em PyTorch (sem biblioteca de RL), disponível no repositório <https://github.com/viniciusrap/projeto-rl>. Treino: Double DQN, *replay buffer* de 50k, ϵ -greedy decaindo até 0,10, *target network*. Três técnicas de domínio aceleram a convergência sem dar a resposta pronta: *action masking* (combo com harmonia $< 1,4$ é bloqueado na cabeça Complementar), pré-treino supervisionado dessa cabeça com a matriz de afinidade, e *curriculum* (cerveja aparece com mais frequência em sex/sáb para o agente *descobrir* a Esquenta de Sexta via recompensa).

```
1 qs = agente.online(estado) # 3 tensores Q
2 mask = env.mask_complementar_valida() # harmonia >= 1.4
3 acao = []
4 for h, q in enumerate(qs):
5     if h == 1: q[-mask] = -1e9 # bloqueia combo
6     acao.append(int(q.argmax()))
7     invalido
```

Code 1. Decodificação da ação (rollout greedy com máscara)

Indicadores avaliados, ligados ao objetivo de lucro: (i) lucro anual estimado e nº de campanhas do calendário gerado; (ii) fração de combos e de **combos inválidos para posto** (alvo: zero); (iii) acerto em 12 cenários canônicos de validação (*sanity*, com *ground truth* de domínio); (iv) cobertura de eventos comerciais; (v) estabilidade da curva de aprendizado (coeficiente de variação).

Evolução metodológica. O modelo final é fruto de ~20 iterações documentadas: cada salto de versão nasceu de uma *falha medida* da anterior. Esse histórico é parte da contribuição — diagnosticar a causa raiz em vez de descartar o método (Tabela 2).

Tabela 2. Evolução: cada versão nasceu de uma falha medida da anterior.

Versão / mudança	Aprendizado
V10 — 6 produtos, recompensa só lucro	Política colapsou numa única ação; o gargalo era a modelagem do MDP, não o algoritmo
V11 — catálogo + datas no estado	F1 por evento correlaciona com cobertura do catálogo ($r \approx 0,75$); <i>feature engineering</i> pesa tanto quanto o algoritmo
V19 — economia explícita (canib., <i>halo</i> , defensivo)	A recompensa precisa modelar o ecossistema econômico, não só “demanda $\times$ desconto”
V20 — regras viram <i>reward shaping</i> + pré-treino da cabeça Combo	<i>Reward shaping</i> puro cai em ótimo local; conhecimento de domínio como inicialização destrava
V21 — rejeitar dados de supermercado	Afinidade de mercearia $\neq$ afinidade de PDV; calibração nativa de balcão

4. Resultados

O modelo final foi treinado com **1 seed $\times$ 5000 episódios**. A Figura 1 mostra a curva de aprendizado: a recompensa média sai de ≈ -14.067 e converge para $\approx +16.354$; a volatilidade local cai **–88%** (coeficiente de variação $3,52 \rightarrow 0,44$), evidência de convergência estável. A política final usa $\approx 56\%$ de turnos com promoção e $\approx 28\%$ de combos — diversificada, usando bem todas as intensidades de desconto e o combo.

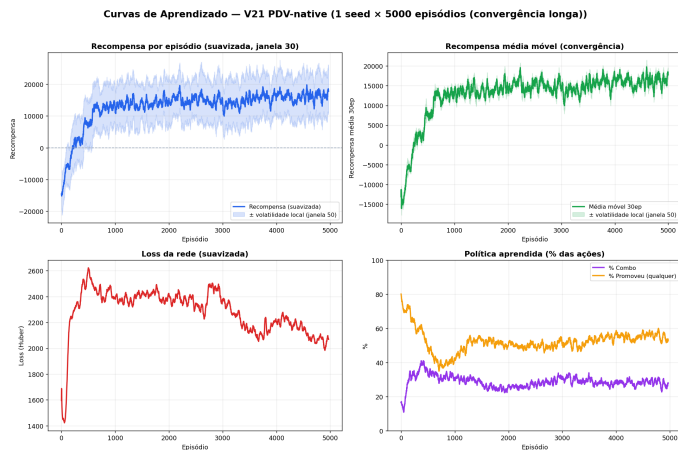


Figura 1. Curvas de aprendizado (1 seed $\times$ 5000 episódios): recompensa por episódio com volatilidade local, média móvel, *loss* da rede e política aprendida (% de combo e de promoção).

Na validação determinística (rollout de 365 dias), o agente acerta 11/12 **cenários canônicos de *sanity*** e gera um calendário anual de 208 campanhas (88 combos + 120 descontos diretos) com **zero combos inválidos para posto** — aprendeu a regra de domínio sem que ela fosse imposta na inferência — e lucro adicional estimado de **R\$ 16.918/ano**.

Combos que o agente descobriu. Os padrões emergiram da recompensa, não de regras fixas, e fazem sentido de varejo: a “**Esquenta de Sexta**” (cerveja + salgadinho em sex/sáb) é o combo nº 1 do ano (~ 45 ocorrências); **café + padaria** domina a manhã de quem para no posto a caminho do trabalho; **Chocolate (caixa)** entra como

presente nas datas comerciais (Mães, Namorados, Pais, Crianças); e nos jogos do Brasil na Copa o agente sobe **cerveja + petisco** (12 jogos cobertos).

Casos difíceis que ele acertou. O valor do modelo aparece onde a decisão é sutil: (i) acerta as **datas-presente** (Dia das Mães, Namorados) — justamente onde versões anteriores falhavam por completo ($F1 = 0$ no V11), pois o sinal histórico desses produtos é raro; (ii) **recusa combos de supermercado** (gelo + whisky) mesmo quando o evento “puxaria” o combo — distinção fina de contexto de balcão aprendida sem regra rígida; (iii) **protege margem**: não dá desconto em produto que já está em alta natural, mas desconta produto de baixo giro para girar estoque — uma regra de varejo não-trivial que emergiu sozinha.

Entrega para o dono. O pipeline gera automaticamente um **calendário visual em HTML** (Figura 2): os 12 meses do ano, cada dia colorido por tipo de campanha (Esquenta de Sexta, jogo da Copa, data comercial, combo, desconto direto), com os destaques do plano e o lucro adicional estimado (R\$ 16.918/ano; 88 combos e 120 descontos diretos). É o produto final — um plano que o dono imprime e segue, com a justificativa de cada campanha.

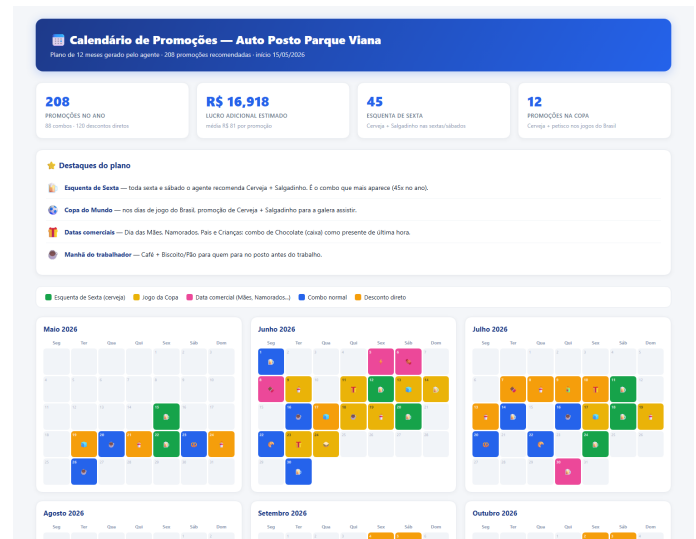


Figura 2. Entregável operacional: calendário visual gerado automaticamente pelo agente — indicadores do ano, destaques do plano e os 12 meses com cada campanha codificada por cor.

5. Considerações finais

O objetivo — um agente que decide promoções para maximizar lucro e reduzir vencimento — foi atingido em termos metodológicos: o *Branching Double DQN* converge de forma estável, produz um calendário operacional íntegro (zero combos inválidos) e faz emergir estratégias de varejo plausíveis sem programá-las explicitamente.

A **limitação central, assumida abertamente**, é de *off-policy evaluation*: o ambiente foi calibrado com seis anos de vendas *sem promoção* e a elasticidade promocional vem da literatura [2], não medida no posto. Portanto os valores em R\$ são **estimativas de simulação**, não medições. Além disso, as decisões do agente são fortemente *induzidas* pelo *reward shaping*, pelo pré-treino e pelo *curriculum* — o que genuinamente emerge é o *timing*, a combinação e a quantidade, não a noção de harmonia em si. O próximo passo necessário é um teste A/B in loco (semanas com e sem o agente) para calibrar a elasticidade real e fechar o ciclo de validação. A contribuição do trabalho é o **método reprodutível e a honestidade calibrada** sobre o que está e o que não está validado por dados.

Referências

- [1] A. Y. Ng, D. Harada e S. Russell, «Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping», em *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.
- [2] T. H. A. Bijmolt, H. J. van Heerde e R. G. M. Pieters, «New Empirical Generalizations on the Determinants of Price Elasticity», *Journal of Marketing Research*, vol. 42, n.º 2, pp. 141–156, 2005.
- [3] V. Mnih et al., «Playing Atari with Deep Reinforcement Learning», *CoRR*, vol. abs/1312.5602, 2013. arXiv: [1312.5602](https://arxiv.org/abs/1312.5602). URL: <http://arxiv.org/abs/1312.5602>
- [4] H. van Hasselt, A. Guez e D. Silver, «Deep Reinforcement Learning with Double Q-Learning», em *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI)*, 2016, pp. 2094–2100. URL: <http://arxiv.org/abs/1509.06461>
- [5] A. Tavakoli, F. Pardo e P. Kormushev, «Action Branching Architectures for Deep Reinforcement Learning», 2018. URL: <http://arxiv.org/abs/1711.08946>